Transformers & BERT Tune

Self-attention and Contextual Encoding

Prof. Surya Prakash

IIT Indore

Introduction

- Many applications require Text processing
- For example,
  - Sentiment analysis
  - Sentence completion
  - Etc.
- Models to process text
  - RNN
  - LSTM
  - GRU
- All these models process text sequentially

Introduction

- Let's revisit sequential processing by RNN

- Consider an example for sentiment analysis
  - Input: The movie was great.
  - Output: sentiment (Positive or negative)

- An RNN reads the sentence **one word at a time**

  The → movie → was → great

- It **remembers previous words** via hidden states h_t.

Introduction

- At each step, it updates its memory (hidden state).

- The meaning builds gradually:
  - "The" → little meaning
  - "The movie" → object
  - "The movie was great" → clear positive sentiment

- Final hidden state is used to predict output.
  - Result: **Positive sentiment**

Introduction: RNN Summary

- RNN is slow and struggles with long dependencies.
  - Example: The movie that I watched last week, despite the slow start and long runtime, was great.
  - To understand **"great"**, the model must remember **"movie"** from far back

- LSTM/GRU are better than simple RNN for long sequences

- However, LSTM/GRU also struggles with very long-range dependencies

Introduction: RNN Summary

- Example sentence
  - "The movie that I watched during my vacation in Goa, which I planned after my exams, along with my friends who came from different cities, despite all the delays and issues, was absolutely fantastic."

- Why does LSTM/GRU struggle?
  - They use **gates** to remember/forget
  - But over **very long sequences**:
    - Important info (like "movie") can still get **diluted**
    - Gates may **forget or overwrite** earlier context

Introduction: Transformer

- Transformer solves the problem of long-range dependencies
  - Processes all words **at once (parallel)**
  - Uses **self-attention** to relate all words
  - **Faster** and handles long dependencies better

Transformer Architecture (Overview)

- The Transformer replaces RNNs/LSTMs with attention mechanisms.

- Key Idea of Transformer:
  - Parallel Processing:
    - Instead of processing sequentially (like in RNN), it processes all words in parallel
  - Self-attention:
    - It learns relationships among the words using attention.

Importance of Self Attention

- Self-attention helps a model focus on important words in a sentence while processing each word.

- Example
  - Sentence: *"The movie was not good"*
  - When understanding **"good"**, the model should also look at **"not"**
  - Self-attention connects them directly
  - So, it understands the sentence is **negative**, not positive

- Why is it useful?
  - Captures relationships between words (near or far)
  - Handles long dependencies easily
  - Works in parallel → faster than RNN/LSTM
  - Improves understanding of **context**

Context Understanding

- **Example:** I went to the bank to deposit money.
- The word **“bank”** can mean:
  - river bank or financial bank
- Without good context
  - Model may be confused about the meaning
- With self-attention
  - “bank” looks at **“deposit”** and **“money”**
  - Clearly understands it is a **financial bank**
- Another example (previous slide)
  - *The movie was not good*
  - “good” attends to **“not”** → correct **negative sentiment**

Context Understanding

- When processing **“bank”**, self-attention assigns
  - **higher importance (weights)** to relevant words like: **“deposit”** and **“money”**
  - and lower importance to less relevant words like “went”, “to”, etc.

- What happens internally
  - “bank” compares itself with all words
  - Finds strong similarity with **finance-related words**
  - Gives them **higher attention scores**

- **Result**
  - Meaning becomes clear: **bank = financial institution (not river side)**

Context Understanding

- "bank" compares itself with all words and finds strong similarity with finance-related words. How?

- What self-attention does
  - Ask a simple question: "Which words in this sentence help me understand *bank*?"

- Step-by-step Analysis
  - "bank" looks at all words: I (not relevant), went (not relevant), deposit (relevant), money (relevant)

- How does it decide, what is relevant?
  - It has **learned it from training data**
  - Words like: "deposit", "money", "loan" → usually related to **financial bank**
  - "river", "water" → usually related to **river bank**
  - So it gives **more importance** to familiar related words

- **So,** "bank" focuses on **deposit + money** and it is understood that it is **financial bank**

Let’s discuss transformer in detail to understand different aspects of it.

Transformer Architecture (Overview)

- Main Components:
  - Input Embedding
  - Positional Encoding
  - Encoder (stack of layers)
  - Decoder (for generation tasks)
  - Self-Attention + Feed Forward layers

Steps to Computer Self-Attention

- **Example Sentence**
  - *“I love AI”*

Step 1: Convert words → embeddings

- **Example Sentence**
  - *"I love AI"*
- Embedding:
  - converting each word toa numerical vector
- Example
  - Assuming small vectors for simplicity:

Word	Embedding
I	[1, 0, 1]
love	[0, 1, 1]
AI	[1, 1, 0]

Step 2: Positional Encoding

- Why Needed?
  - Transformer sees input like: [I, love, AI] = unordered set
- How Encoding Works
  - Each position gets unique pattern:

Position	Encoding (simplified)
1	[0.1,0.2,0.3]
2	[0.91, -0.42,0.3]
3	[0.5,0.6,−0.2]

- Example
  - Original embedding: “love” = [0,1,1]
  - After adding position: [0,1,1] + [0.91, -0.42,0.3]
  - Now model knows: “love is in middle”
- Insight:
  - Nearby positions → similar encodings
  - Far positions → different encodings

Step 2: Word + Positional Encoding

Word	Position	Word Embedding	Positional Encoding	Calculation (element-wise addition)	Final Embedding
I	1	[1, 0, 1]	[0.1, 0.2, 0.3]	[1+0.1, 0+0.2, 1+0.3]	[1.1, 0.2, 1.3]
love	2	[0, 1, 1]	[0.91, -0.42, 0.3]	[0+0.91, 1+(-0.42), 1+0.3]	[0.91, 0.58, 1.3]
AI	3	[1, 1, 0]	[0.5, 0.6, -0.2]	[1+0.5, 1+0.6, 0+(-0.2)]	[1.5, 1.6, -0.2]

Step 3: Create Q, K, V (learned weights)

- We multiply embeddings (*X*) with matrices:
  - Q = XW_q
  - K = XW_k
  - V = XW_v

- Assume:
  - $W_q = W_k = W_v =$ *identity (for simplicity)*

- So:
  - Q = K = V = embeddings

Step 3: Create Q, K, V (learned weights)

Matrix	Role / Intuition	Effect on Word Embedding	Example for "bank"
W_Q (Query)	Prepares the word to ask: "Which words are relevant to me?"	Transforms embedding into a vector that **searches for related words**	Q_{bank} looks at $K_{deposit}$ and K_{money} to decide they are important
W_K (Key)	Prepares each word to say: "What information do I contain?"	Transforms embedding into a vector used for **matching with Queries**	K_{money} indicates it is finance-related, K_{river} indicates water-related
W_V (Value)	Prepares each word to give meaning if selected	Transforms embedding into a vector that is **passed to the output** carrying actual info	V_{money} = [8,1] passes financial info to "bank" vector after attention
W_O (Output, after multi-head)	Combines multiple heads into a final embedding	Produces the **final context-aware embedding**	Final "bank" embedding = [3.3, 1.75], strongly finance-related

Step 4: Compute attention scores ($Q \times K^T$)

- Score$(i, j) = Q_i \cdot K_j$

- For word **"I"**:
  - I·I = 2.94
  - I·love = 2.80
  - I·AI = 1.71

- So, scores:
  - [2.94, 2.80, 1.71]

Comparison	v_I Elements	v_X Elements	Element-wise Product	Sum (Score)
I vs I	[1.1, 0.2, 1.3]	[1.1, 0.2, 1.3]	[1.1×1.1, 0.2×0.2, 1.3×1.3] = [1.21, 0.04, 1.69]	1.21 + 0.04 + 1.69 = 2.94
I vs love	[1.1, 0.2, 1.3]	[0.91, 0.58, 1.3]	[1.1×0.91, 0.2×0.58, 1.3×1.3] = [1.001, 0.116, 1.69]	1.001 + 0.116 + 1.69 = 2.807
I vs AI	[1.1, 0.2, 1.3]	[1.5, 1.6, -0.2]	[1.1×1.5, 0.2×1.6, 1.3×(-0.2)] = [1.65, 0.32, -0.26]	1.65 + 0.32 - 0.26 = 1.71

Step 5: Scale + Softmax

- Scaling of Score by dividing the obtained value of by QK^T by $\sqrt{d_k}$

$$\frac{QK^T}{\sqrt{d_k}} \ (here\ d_k = 3)$$

- So, scaled scores:
  - Before: [2.94, 2.80, 1.71]
  - After: $\left[\frac{2.94}{\sqrt{3}}, \frac{2.80}{\sqrt{3}}, \frac{1.71}{\sqrt{3}}\right]$

- After applying softmax to scaled weights, approx. weights:
  - [1.698, 1.62, 0.987]

Step 6: Weighted sum of V

$$\text{Output} = \sum (\textit{weights} \times V)$$

- We have: v_I=[1.1,0.2,1.3],v_{love}=[0.91,0.58,1.3],v_{AI}=[1.5,1.6,−0.2]
- For “I”:
  - Output = 1.698 × [1.1, 0.2, 1.3] + 1.62×[0.91,0.58,1.3] + 0.987 ×[1.5,1.6,−0.2]
  - = [4.823,2.858,4.116]

- Final vector ≈ **context-aware representation of “I”:** $v_{I,context}$≈[4.823,2.858,4.116]

- **Key Insight:**
  - Each word becomes: “a mixture of all words based on importance”

3. Multi-Head Attention

- Multi-head attention lets the model look at the sentence in multiple ways simultaneously and combine the insights.

- **Why multiple heads?**
  - Each head learns a **different type of relationship**:
    - Head 1 → position/context - structural connection
    - Head 2 → meaning (bank–money) - semantic connection
    - Head 3 → grammar (subject–verb)

3. Multi-Head Attention

- **Main idea:**
  - Same sentence, multiple “views”
- **How it works**
  - Each head has its own: W_Q, W_K, W_V
  - Each head computes attention separately
  - Outputs are **combined (concatenated)**
- **Benefit**
  - Captures **multiple relationships at once**
  - More powerful than single attention

4. Transformer Encoder Layer

- Each encoder layer has:
  - **1. Multi-head attention**
  - **2. Add & Normalize**
  - **3. Feed Forward Network**
  - **4. Add & Normalize**

- **Example Flow**
  - Input: "I love AI"
- **Step 1: Self-attention**
  - Each word updates using all words
- **Step 2: Add & Normalize**
  - Output $= LayerNorm\big(X + Attention(X)\big)$
  - Prevents vanishing/exploding
- **Step 3: Feed Forward Network**
  - $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$
  - Adds non-linearity
- **Step 4: Repeat layers**
  - Stack multiple encoders (e.g., 12 in BERT)

1. First transformation

$$xW_1 + b_1$$

→ Takes input vector $x$and **changes its representation**
→ Usually increases dimension (e.g., 512 → 2048)

2. Activation (ReLU)

$$\max(0, \cdot)$$

→ Keeps positive values, makes negatives **0**
Example:

$$[-, 2, \ 3 \ \ -1] \rightarrow [0, \ , 3 \ 0]$$

→ Adds **non-linearity** (very important)

3. Second transformation

$$(\cdot)W_2 + b_2$$

→ Brings vector back to original size
→ Refines the representation

Thank You